



**Instituto Tecnológico De Tijuana**  
**Departamento De Sistemas Y Computación**  
**Ingeniería En Sistemas Computacionales**

**Patrones de desarrollo de software**

**Unidad:**

Unidad 5

**Tema:**

Patrón “Estado”

**Integrantes:**

Hernandez Ramirez Lixbeth Denisse - 22210313

**Docente:**

MARIBEL GUERRERO LUIS

Tijuana, B.C.

04 de Mayo Del 2026

## ÍNDICE

|                                   |           |
|-----------------------------------|-----------|
| <b>INTRODUCCIÓN.....</b>          | <b>3</b>  |
| <b>OBJETIVO.....</b>              | <b>3</b>  |
| <b>DIAGRAMA UML.....</b>          | <b>4</b>  |
| <b>DIAGRAMA DE ESTADO.....</b>    | <b>4</b>  |
| <b>CÓDIGO EN EJECUCIÓN.....</b>   | <b>5</b>  |
| <b>CÓDIGO.....</b>                | <b>5</b>  |
| Programs.cs.....                  | 5         |
| IEstado.cs.....                   | 6         |
| MaquinaContexto.cs.....           | 6         |
| EnEsperaState.cs.....             | 7         |
| ValidaFuncionState.cs.....        | 8         |
| SeleccionProductoState.cs.....    | 9         |
| CreditoDisponibleState.cs.....    | 9         |
| EsperaCancelacionState.cs.....    | 10        |
| ValidacionCompraState.cs.....     | 11        |
| ProcesoEntregaState.cs.....       | 12        |
| EntregaProductoState.cs.....      | 12        |
| DevolverCreditoState.cs.....      | 13        |
| ProductoNoDisponibleState.cs..... | 14        |
| AgregarCreditoState.cs.....       | 14        |
| <b>CONCLUSIÓN.....</b>            | <b>16</b> |
| <b>REFERENCIA.....</b>            | <b>17</b> |

# State

---

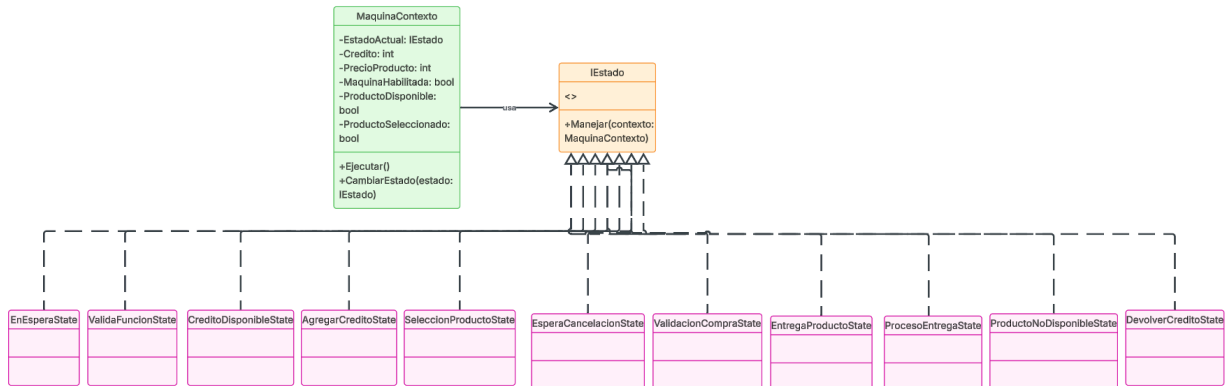
## INTRODUCCIÓN

State es un patrón de diseño de comportamiento que permite a un objeto alterar su comportamiento cuando su estado interno cambia. Parece como si el objeto cambiará su clase. La idea principal es que, en cualquier momento dado, un programa puede encontrarse en un número finito de estados. Dentro de cada estado único, el programa se comporta de forma diferente y puede cambiar de un estado a otro instantáneamente. Sin embargo, dependiendo de un estado actual, el programa puede cambiar o no a otros estados. Estas normas de cambio llamadas transiciones también son finitas y predeterminadas. Las máquinas de estado se implementan normalmente con muchos operadores condicionales (if o switch) que seleccionan el comportamiento adecuado dependiendo del estado actual del objeto. Normalmente, este “estado” es tan solo un grupo de valores de los campos del objeto.

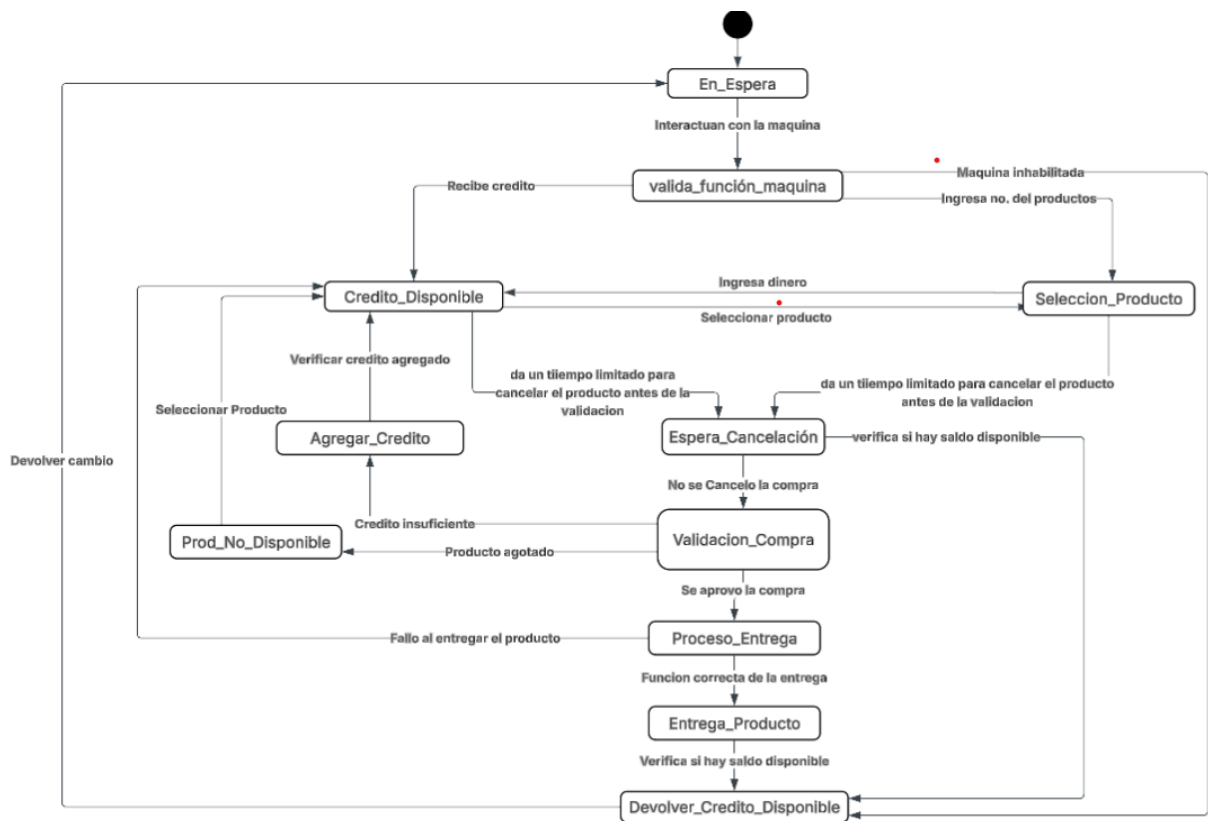
## OBJETIVO

Desarrollar un sistema que simule los estados por los que pasa una máquina expendedora y que es lo que pasa en las diferentes situaciones de los estados.

## DIAGRAMA UML



## DIAGRAMA DE ESTADO



## CÓDIGO EN EJECUCIÓN

```
-----
❏ Estado: En espera
-----

-----
❏ Estado: Validación de máquina
-----

Máquina lista
1. Insertar dinero
2. Seleccionar producto
1
-----

❏ Estado: Agregar crédito
-----

Ingrese la cantidad a agregar:
15
-----

❏ Estado: Crédito disponible
-----

Crédito actual: 15
1. Insertar crédito
2. Seleccionar producto
3. Cancelar
2
-----

❏ Estado: Selección de producto
-----

-----
❏ Estado: Espera de cancelación
-----

1. Continuar
2. Cancelar
1
-----

❏ Estado: Validación de compra
-----

Crédito insuficiente
-----

❏ Estado: Agregar crédito
-----

Ingrese la cantidad a agregar:
15
-----

❏ Estado: Espera de cancelación
-----

1. Continuar
2. Cancelar
1
-----

❏ Estado: Validación de compra
-----

-----

❏ Estado: Proceso de entrega
-----

-----

-----
❏ Estado: Entrega de producto
-----

-----

❏ Estado: Devolver crédito
-----

Cambio devuelto: 10
-----

❏ Estado: En espera
-----

-----

❏ Estado: Validación de máquina
-----

Máquina lista
1. Insertar dinero
2. Seleccionar producto
```

## CÓDIGO

### Programs.cs

```
using System;

namespace MaquinaE
{
    /*
        Punto de entrada del programa.
        Inicializa el contexto y ejecuta el ciclo continuo.
    */
    class Program
    {
```

```

        static void Main()
        {
            MaquinaContexto maquina = new MaquinaContexto(new EnEsperaState());

            while (true)
            {
                maquina.Ejecutar();
            }
        }
    }
}

```

## **IEstado.cs**

```

namespace MaquinaE
{
    /*
        Interfaz del patrón State.
        Obliga a todos los estados a implementar el método Manejar,
        que contiene el comportamiento del estado y decide la transición.
    */
    public interface IEstado
    {
        void Manejar(MaquinaContexto contexto);
    }
}

```

## **MaquinaContexto.cs**

```

using System;

namespace MaquinaE
{
    /*
        Contexto del patrón State.

        Contiene:
        - El estado actual
        - Los datos compartidos entre estados
        - La lógica para cambiar de estado

        Además guarda información clave como:
        - Crédito
        - Si el usuario ya seleccionó un producto
    */
}

```

```

public class MaquinaContexto
{
    public IEstado EstadoActual { get; set; }

    public int Credito { get; set; } = 0;
    public int PrecioProducto { get; set; } = 20;

    public bool MaquinaHabilitada { get; set; } = true;
    public bool ProductoDisponible { get; set; } = true;

    // Variable clave para mantener el flujo de compra
    public bool ProductoSeleccionado { get; set; } = false;

    public MaquinaContexto(IEstado estadoInicial)
    {
        EstadoActual = estadoInicial;
    }

    public void Ejecutar()
    {
        EstadoActual.Manejar(this);
    }

    public void CambiarEstado(IEstado nuevoEstado)
    {
        EstadoActual = nuevoEstado;
    }
}

```

## EnEsperaState.cs

```

using System;

namespace MaquinaE
{
    /*
     * Estado inicial del sistema.
     * Representa la máquina sin interacción activa.
     */
    public class EnEsperaState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)
        {
            Console.WriteLine("\n-----");
            Console.WriteLine("\n► Estado: En espera\n");
        }
    }
}

```

```

        Console.WriteLine("-----\n");

        contexto.CambiarEstado(new ValidaFuncionState());
    }
}

```

## ValidaFuncionState.cs

```

using System;

namespace MaquinaE
{
    /*
     * Verifica si la máquina está disponible y permite iniciar interacción.
     */
    public class ValidaFuncionState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)
        {
            Console.WriteLine("\n-----");
            Console.WriteLine("\n► Estado: Validación de máquina\n");
            Console.WriteLine("-----\n");

            if (!contexto.MaquinaHabilitada)
            {
                Console.WriteLine("Máquina inhabilitada");
                contexto.CambiarEstado(new EnEsperaState());
                return;
            }

            Console.WriteLine("Máquina lista");

            Console.WriteLine("1. Insertar dinero");
            Console.WriteLine("2. Seleccionar producto");

            if (!int.TryParse(Console.ReadLine(), out int op))
            {
                Console.WriteLine("Entrada inválida");
                return;
            }

            if (op == 1)
                contexto.CambiarEstado(new AgregarCreditoState());
            else if (op == 2)
                contexto.CambiarEstado(new SeleccionProductoState());
        }
    }
}

```



```

    }
}
}

```

## SeleccionProductoState.cs

```

using System;

namespace MaquinaE
{
    /*
     * El usuario elige producto.
     * Si no hay dinero, lo redirige a agregar crédito.
     */
    public class SeleccionProductoState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)
        {
            Console.WriteLine("\n-----");
            Console.WriteLine("\n► Estado: Selección de producto\n");
            Console.WriteLine("-----\n");

            contexto.ProductoSeleccionado = true;

            if (contexto.Credito <= 0)
            {
                Console.WriteLine("Debe ingresar crédito primero");
                contexto.CambiarEstado(new CreditoDisponibleState());
                return;
            }

            contexto.CambiarEstado(new EsperaCancelacionState());
        }
    }
}

```

## CreditoDisponibleState.cs

```

using System;

namespace MaquinaE
{
    /*
     * Estado donde el usuario tiene crédito disponible
     * y puede decidir qué hacer.
     */
}

```

```

*/
public class CreditoDisponibleState : IEstado
{
    public void Manejar(MaquinaContexto contexto)
    {
        Console.WriteLine("\n-----");
        Console.WriteLine("\n► Estado: Crédito disponible\n");
        Console.WriteLine("-----\n");
        Console.WriteLine($"Crédito actual: {contexto.Credito}");

        Console.WriteLine("1. Insertar crédito");
        Console.WriteLine("2. Seleccionar producto");

        if (contexto.Credito > 0)
            Console.WriteLine("3. Cancelar");

        if (!int.TryParse(Console.ReadLine(), out int op))
        {
            Console.WriteLine("Entrada inválida");
            return;
        }

        if (op == 1)
            contexto.CambiarEstado(new AgregarCreditoState());
        else if (op == 2)
            contexto.CambiarEstado(new SeleccionProductoState());
        else if (op == 3 && contexto.Credito > 0)
            contexto.CambiarEstado(new DevolverCreditoState());
        else
            Console.WriteLine("Opción no válida");
    }
}
}

```

## EsperaCancelacionState.cs

```

using System;

namespace MaquinaE
{
    /*
     * Permite cancelar antes de validar la compra.
     */
    public class EsperaCancelacionState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)

```

```

    {
        Console.WriteLine("\n-----");
        Console.WriteLine("\n► Estado: Espera de cancelación\n");
        Console.WriteLine("-----\n");

        Console.WriteLine("1. Continuar");
        Console.WriteLine("2. Cancelar");

        if (!int.TryParse(Console.ReadLine(), out int op))
        {
            Console.WriteLine("Entrada inválida");
            return;
        }

        if (op == 2)
            contexto.CambiarEstado(new DevolverCreditoState());
        else
            contexto.CambiarEstado(new ValidacionCompraState());
    }
}

```

## ValidacionCompraState.cs

```

using System;

namespace MaquinaE
{
    /*
     * Valida crédito y disponibilidad.
     */
    public class ValidacionCompraState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)
        {
            Console.WriteLine("\n-----");
            Console.WriteLine("\n► Estado: Validación de compra\n");
            Console.WriteLine("-----\n");

            if (!contexto.ProductoDisponible)
            {
                contexto.CambiarEstado(new ProductoNoDisponibleState());
                return;
            }

            if (contexto.Credito < contexto.PrecioProducto)

```

```

        {
            Console.WriteLine("Crédito insuficiente");
            contexto.CambiarEstado(new AgregarCreditoState());
            return;
        }

        contexto.CambiarEstado(new ProcesoEntregaState());
    }
}

```

## ProcesoEntregaState.cs

```

using System;

namespace MaquinaE
{
    /*
     * Simula la entrega del producto.
     */
    public class ProcesoEntregaState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)
        {
            Console.WriteLine("\n-----");
            Console.WriteLine("\n► Estado: Proceso de entrega\n");
            Console.WriteLine("-----\n");

            bool fallo = false;

            if (fallo)
                contexto.CambiarEstado(new DevolverCreditoState());
            else
                contexto.CambiarEstado(new EntregaProductoState());
        }
    }
}

```

## EntregaProductoState.cs

```

using System;

namespace MaquinaE
{
    /*

```

```

        Entrega el producto y descuenta crédito.
    */
    public class EntregaProductoState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)
        {
            Console.WriteLine("\n-----");
            Console.WriteLine("\n► Estado: Entrega de producto\n");
            Console.WriteLine("-----\n");

            contexto.Credito -= contexto.PrecioProducto;
            contexto.ProductoSeleccionado = false;

            contexto.CambiarEstado(new DevolverCreditoState());
        }
    }
}

```

## DevolverCreditoState.cs

```

using System;

namespace MaquinaE
{
    /*
        Devuelve el dinero restante.
    */
    public class DevolverCreditoState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)
        {
            Console.WriteLine("\n-----");
            Console.WriteLine("\n► Estado: Devolver crédito\n");
            Console.WriteLine("-----\n");

            if (contexto.Credito > 0)
            {
                Console.WriteLine($"Cambio devuelto: {contexto.Credito}");
                contexto.Credito = 0;
            }

            contexto.ProductoSeleccionado = false;

            contexto.CambiarEstado(new EnEsperaState());
        }
    }
}

```

```
}
```

## **ProductoNoDisponibleState.cs**

```
using System;

namespace MaquinaE
{
    /*
     * Maneja el caso donde no hay producto disponible.
     */
    public class ProductoNoDisponibleState : IEstado
    {
        public void Manejar(MaquinaContexto contexto)
        {
            Console.WriteLine("\n-----");
            Console.WriteLine("\n► Estado: Producto no disponible\n");
            Console.WriteLine("-----\n");

            Console.WriteLine("1. Elegir otro");
            Console.WriteLine("2. Cancelar");

            if (!int.TryParse(Console.ReadLine(), out int op))
            {
                Console.WriteLine("Entrada inválida");
                return;
            }

            if (op == 1)
                contexto.CambiarEstado(new SeleccionProductoState());
            else
                contexto.CambiarEstado(new DevolverCreditoState());
        }
    }
}
```

## **AgregarCreditoState.cs**

```
using System;

namespace MaquinaE
{
    /*
     * Permite ingresar dinero a la máquina.
     * Mantiene la intención del usuario.
     */
}
```

```

*/
public class AgregarCreditoState : IEstado
{
    public void Manejar(MaquinaContexto contexto)
    {
        Console.WriteLine("\n-----");
        Console.WriteLine("\n► Estado: Agregar crédito\n");
        Console.WriteLine("-----\n");
        Console.WriteLine("\nIngrese la cantidad a agregar:");

        if (!int.TryParse(Console.ReadLine(), out int dinero))
        {
            Console.WriteLine("Entrada inválida");
            return;
        }

        contexto.Credito += dinero;

        // Si ya había intención de compra, continúa el flujo
        if (contexto.ProductoSeleccionado)
            contexto.CambiarEstado(new EsperaCancelacionState());
        else
            contexto.CambiarEstado(new CreditoDisponibleState());
    }
}

```

## **CONCLUSIÓN**

La realización del sistema de una máquina expendedora mediante el uso del patrón de diseño State permitió establecer el comportamiento del sistema de manera organizada y haciéndolo fácil de mantener. A través de la implementación de una interfaz común y múltiples estados concretos, y esto evitó estructuras condicionales complejas y mejoro la claridad del código.



## REFERENCIA

*State - Patrones de comportamiento.* (n.d.). Refactoring.Guru. Retrieved May 4, 2026, from <https://refactoring.guru/es/design-patterns/state>